

Trabalho Pratico 2 - Organização de Computadores

Ingrid Luiza Rodrigues Silva - 5938
Otávio Francisco Sabino Tavares - 5912

¹ Universidade Federal de Viçosa - Campus Florestal

ingrid.r.silva@ufv.br, otavio.sabino@ufv.br

1. Introdução

O RISC-V é uma arquitetura baseada em conjunto reduzido de instruções (RISC), amplamente utilizada para estudos acadêmicos devido à sua simplicidade e modularidade. Neste trabalho foram desenvolvidas as duas etapas do projeto proposto na disciplina: a primeira correspondente à criação de um testbench para simulação do caminho de dados do processador, e a segunda correspondente à implementação e validação desse mesmo caminho de dados em uma placa de FPGA.

O objetivo principal deste trabalho foi validar o funcionamento das instruções atribuídas ao grupo 21, primeiro acompanhando o estado dos registradores em ambiente de simulação e, em seguida, observando esse mesmo comportamento diretamente em hardware, por meio de displays de 7 segmentos.

As instruções implementadas pelo grupo foram: lw, sw, add, xor, addi, sll e bne e o repositório pode ser acessado em: <https://github.com/tavinEscada/tp2oc>

2. Desenvolvimento

2.1. Módulo PC

O Program Counter é um registrador de 32 bits responsável por armazenar a instrução que está sendo executada; a cada ciclo do clock ele é atualizado com o endereço da próxima instrução. Ele é síncrono e só muda de valor quando o clock sobe, fazendo com que o caminho só trabalhe com o mesmo endereço durante um ciclo inteiro.

```
module pc(  
    clock,  
    reset,  
    entrada,  
    saida  
);  
  
input wire reset, clock;  
input wire [31:0] entrada;  
output reg [31:0] saida;
```

Figure 1. Portas do módulo PC.

A entrada vem do multiplexador branch, que decide entre PC + 4 (execução sequencial) e PC + imediato (desvio do BNE) e a saída vai para a memória de instruções.

```

always @(posedge clock) begin
if(reset) begin
    saida <= 32'b00000000000000000000000000000000;
end
else begin
    saida <= entrada;
end
end

```

Figure 2. Funcionamento do módulo.

2.2. Módulo memória de instruções

A memória de instruções armazena o programa a ser executado, dado um endereço do PC. Ela então retorna a instrução de 32 bits a partir do endereço para o resto do caminho.

```

module memoriaInstrucoes(
    endereco,
    instrucao
);

input wire [31:0] endereco;
output reg [31:0] instrucao;

reg [31:0] memInst [0:63];

always @(*) begin
    instrucao = memInst[endereco/4];
end

endmodule

```

Figure 3. Módulo de memória de instruções.

Dentro do laço always existe a divisão por 4 devido ao fato de que temos um endereço em bytes, mas cada instrução ocupa 32 bits (4 bytes). O endereço da entrada vem do PC e a instrução vai para o controle.

2.3. Módulo control

O controle recebe, vindos da memória de instruções, os 7 bits de opcode e decide como os outros módulos devem se comportar, usando os elementos listados abaixo.

```

module control(
    opcode,
    branch,
    memread,
    memtoreg,
    aluop,
    memwrite,
    alusrc,
    regwrite
);

    input wire [6:0] opcode;
    output reg branch;
    output reg memread;
    output reg memtoreg;
    output reg [1:0] aluop;
    output reg memwrite;
    output reg alusrc;
    output reg regwrite;

```

Figure 4. Portas do módulo.

Para cada instrução ocorre uma atribuição de valores para as portas. Por exemplo, a instrução lw, caracterizada por '0000011', vai gerar a seguinte atribuição:

```

7'b0000011: begin // lw
    branch <= 0;
    memread <= 1;
    memtoreg <= 1;
    aluop <= 2'b00;
    memwrite <= 0;
    alusrc <= 1;
    regwrite <= 1;
end

```

Figure 5. Atribuições para a instrução lw.

2.4. Módulo banco de registradores

O banco de registradores é o conjunto dos 32 registradores nos quais o processador armazena valores temporários durante a execução. O principal desse módulo é a escrita e a leitura:

```

always @(posedge clk) begin
    if(rw == 1 && wreg != 5'b00000) begin
        registradores[wreg] <= wd;
    end
end

always @(*) begin
    rd1 = registradores[r1];
    rd2 = registradores[r2];
end

```

Figure 6. Escrita e leitura.

Note que a escrita só acontece na borda de subida do clock e apenas se $rw = 1$ (controle autorizou a escrita) e $wreg \neq 0$ (valor de x0 não é sobrescrito). Já a leitura é imediata, e qualquer mudança em $r1$ ou $r2$ reflete instantaneamente em $rd1$ e $rd2$, sem interferência do clock.

2.5. Gerador de imediato

Esta etapa representa a extração dos bits do imediato espalhados pelos 32 bits da instrução, remontando-os na ordem certa e fazendo a extensão de sinal. Cada tipo de instrução codifica o imm em posições diferentes, então o módulo usa o opcode para remontar da forma certa. Um exemplo são as instruções de tipo I `lw` e `addi`:

```

7'b0000011: //lw
    imediato <= {{20{instrucao[31]}}, instrucao[31:20]};

```

Figure 7. Atribuição de lw e addi.

A extensão de sinal é feita para preservar o sinal negativo nos casos em que o bit mais significativo do imediato for 1, completando os bits extras com 1s, e quando o bit mais significativo for 0, completando os bits extras com 0s.

2.6. Módulo Mux

Basicamente, temos um multiplexador 2x1 de 32 bits e ele seleciona qual valor será o segundo operando da ALU: o valor de $rs2$ vindo do banco de registradores ou o imediato gerado. Ele é combinacional, mudando instantaneamente acompanhando as entradas.

2.7. ULA (ou ALU control)

Este é o decodificador da operação da ALU; o módulo de controle abordado anteriormente só sabe o tipo de instrução (pela `aluOp`) mas sem a operação exata. Já o ALU Control detalha essa informação usando o `funct3` e `funct7` para gerar o código para a ALU executar.

2.8. ALU

Essa já é a unidade que executa as operações aritméticas e lógicas do processador. Ela recebe dois operandos de 32 bits e o código de operação vindo do ALU Control, produz o resultado e uma flag zero usada pelo BNE para decidir se desvia. Um exemplo de funcionamento é o `add`:

```

4'b0010: begin //ADD add, addi, lw, sw
    aluResult <= data1 + data2;
    zero <= 0;

```

Figure 8. Aplicação para instrução add.

2.9. Memória de dados

Essa é a etapa de leitura e escrita do processador, usada pelas instruções lw e sw. A memória de instruções só lê palavras de 32 bits, enquanto essa opera com endereçamento em bytes, de forma a ser possível acessar posições individuais em uma palavra de 32 bits. A parte mais importante dessa etapa é o endereçamento em bytes:

```

always @(*)begin
    if(memread == 1)begin
        //24 no byte de acordo com o endereço
        case (endereço[1:0])
            2'b00: rd <= {24'b0, memória[endereço >> 2][31:24]};
            2'b01: rd <= {24(memória[endereço >> 2][23]), memória[endereço >> 2][23:16]};
            2'b10: rd <= {24(memória[endereço >> 2][15]), memória[endereço >> 2][15:8]};
            2'b11: rd <= {24(memória[endereço >> 2][7]), memória[endereço >> 2][7:0]};
        endcase
    end
end

always @(negedge clk)begin
    if(memwrite == 1)begin
        //escreve no byte
        case (endereço[1:0])
            2'b00: memória[endereço >> 2][31:24] <= wd[7:0];
            2'b01: memória[endereço >> 2][23:16] <= wd[7:0];
            2'b10: memória[endereço >> 2][15:8] <= wd[7:0];
            2'b11: memória[endereço >> 2][7:0] <= wd[7:0];
        endcase
    end
end

```

Figure 9. Funcionamento do endereçamento.

Sem a lógica acima, comandos lw e sw em endereços não múltiplos de 4 acasariam o dado errado, corrompendo o funcionamento.

2.10. Implementação do Testbench

Para a realização da primeira etapa do trabalho foi desenvolvido um testbench responsável pela simulação do caminho de dados do processador RISC-V.

O principal objetivo do testbench foi executar as instruções atribuídas ao grupo 21 e exibir no terminal o estado dos 32 registradores durante a execução da simulação.

O testbench foi responsável por gerar automaticamente os sinais de clock e reset utilizados pelo processador, permitindo que as instruções fossem executadas de forma contínua durante a simulação.

2.11. Execução das Instruções

Para validar o funcionamento do processador foram utilizados programas de teste escritos em Assembly e convertidos para binário.

Durante a execução, as instruções eram buscadas na memória de instruções e processadas pelo caminho de dados implementado.

As instruções lw e sw foram utilizadas para operações de leitura e escrita na memória de dados. Já as instruções add, xor e addi realizaram operações aritméticas e lógicas entre registradores.

Também foram realizados testes utilizando a instrução sll, responsável pelo deslocamento lógico de bits para a esquerda, e a instrução bne, utilizada para realizar desvios condicionais durante a execução do programa.

2.12. Exibição dos Registradores

Um dos requisitos principais da Parte 1 do trabalho foi a impressão do conteúdo dos 32 registradores durante ou ao final da execução da simulação.

Para isso, foram adicionados comandos no testbench capazes de exibir no terminal os valores armazenados em cada registrador após a execução das instruções.

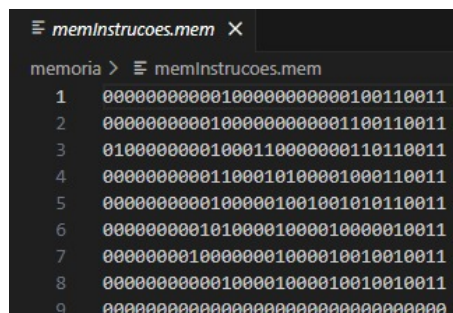
Essa etapa foi importante para validar o funcionamento correto do processador, permitindo verificar se os valores produzidos pelas instruções correspondiam aos resultados esperados.

2.13. Resultado

Os resultados obtidos durante a simulação demonstraram o correto funcionamento das instruções implementadas no processador. As imagens apresentadas mostram o conteúdo da memória de instruções contendo os códigos binários utilizados nos testes realizados pelo grupo.

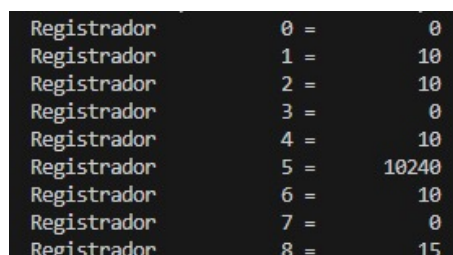
Durante a execução do testbench foi possível observar o processamento das instruções e as alterações ocorrendo nos registradores conforme esperado para cada operação executada. As instruções de soma, operações lógicas, deslocamento de bits, leitura e escrita em memória e desvios condicionais apresentaram comportamento compatível com a arquitetura RISC-V.

Além disso, a utilização dos arquivos de memória permitiu organizar melhor os testes e facilitar a validação das instruções implementadas. A análise das saídas geradas durante a simulação ajudou na identificação de erros e na verificação do funcionamento correto do caminho de dados desenvolvido.



```
memInstrucoes.mem X
memoria > memInstrucoes.mem
1 00000000001000000000100110011
2 000000000010000000001100110011
3 01000000001000110000000110110011
4 00000000001100010100001000110011
5 00000000001000001001001010110011
6 000000000010100001000010000010011
7 0000000000100000001000010010010011
8 00000000000100001000010010010011
9 00000000000000000000000000000000
```

Figure 10. Entrada 1.



```
Registrador 0 = 0
Registrador 1 = 10
Registrador 2 = 10
Registrador 3 = 0
Registrador 4 = 10
Registrador 5 = 10240
Registrador 6 = 10
Registrador 7 = 0
Registrador 8 = 15
```

Figure 11. Saída correspondente.

```

memInstrucoes.mem M X
memoria > memInstrucoes.mem
1  00000000000000000000000000000000
2  000000000001000000000000100110011
3  0000000000010000000000001100110011
4  010000000001000110000000110110011
5  00000000001100010100001000110011
6  00000000001000010101000000110011
7  00100000001000001010000000100011

```

Figure 12. Entrada 2

```

Registrador    0 =    0
Registrador    1 =   10
Registrador    2 =   10
Registrador    3 =    0
Registrador    4 =   10
Registrador    5 =    0
Registrador    6 =   10
Registrador    7 =    0

```

Figure 13. Saída correspondente.

```

memInstrucoes.mem M X
memoria > memInstrucoes.mem
1  00100000001000001010000000100011
2  001010000010100001001000001100011
3  00000000001100001000000010010011

```

Figure 14. Entrada 3

```

Registrador    0 =    0
Registrador    1 =   10
Registrador    2 =    1
Registrador    3 =    0

```

Figure 15. Saída correspondente.

2.14. Instruções de Execução

Para compilar o testbench, é usado o comando de terminal 'iverilog -o sim/cd testbench/tb.v'. Já para executar o testbench, usa-se 'vvp sim/cd'.

3. Implementação em Hardware (FPGA)

Na segunda etapa do trabalho, o caminho de dados validado por simulação na Parte 1 foi adaptado para execução física em uma placa de FPGA da família Cyclone IV E (dispositivo EP4CE115F29C7), utilizando o software Quartus para síntese, ajuste de pinos (*pin planning*) e programação do dispositivo. O principal desafio dessa etapa foi substituir a observação dos registradores via terminal, usada na simulação, por uma interface de depuração física baseada em displays de 7 segmentos, LEDs e botões da placa.

3.1. Módulo top-level (cd)

Para a síntese em FPGA, foi criado um módulo de topo (cd) que instancia e interliga todos os módulos desenvolvidos na Parte 1 (PC, memória de instruções, banco de registradores, gerador de imediato, controle, ALU Control, ALU, memória de dados e os multiplexadores), expondo como portas apenas os sinais de entrada e saída que se conectam fisicamente à placa: botões, switches, displays de 7 segmentos e LEDs.

```
module cd(  
    input wire clk, //BOTAO-0: clock  
    input wire reset, //BOTAO-1: reset  
    input wire btn_reg, //BOTAO-2: avança índice do registrador  
    input wire sw_retro, //SW1: retrocede índice do registrador  
  
    output wire [6:0] HEX0, //valor do registrador [3:0]  
    output wire [6:0] HEX1, //valor do registrador [7:4]  
    output wire [6:0] HEX2, //valor do registrador [11:8]  
    output wire [6:0] HEX3, //valor do registrador [15:12]  
    output wire [6:0] HEX4, //índice do registrador [3:0]  
    output wire [6:0] HEX5, //índice do registrador [7:4]  
    output wire [6:0] HEX6, //pc [3:0]  
    output wire [6:0] HEX7, //pc [7:4]  
  
    output wire [7:0] LEDG //índice do registrador em binário  
);
```

Figure 16. Portas do módulo top-level cd.

Diferentemente da simulação, em que o clock e o reset eram gerados automaticamente pelo testbench, na placa esses sinais são acionados manualmente pelo usuário através de botões físicos (BOTAO-0 para clock e BOTAO-1 para reset), permitindo executar o processador instrução por instrução e observar o efeito de cada ciclo isoladamente.

3.2. Inversão dos sinais de entrada

Os botões da placa utilizada são ativos em nível baixo, ou seja, retornam 0 quando pressionados e 1 quando soltos. Como o restante do caminho de dados foi projetado considerando lógica ativa em nível alto (clock sobe ao ser acionado, reset ativo em 1), foi necessário inverter esses sinais antes de utilizá-los internamente:

```
wire clk_int = ~clk; // borda de subida quando botão é pressionado  
wire reset_int = ~reset; // 1 quando pressionado  
wire btn_reg_int = ~btn_reg;
```

Figure 17. Inversão dos sinais dos botões.

Dessa forma, o clock interno (clk_int) sobe no instante em que o botão físico é pressionado, preservando a borda de subida esperada pelos módulos PC e banco de registradores sem que fosse necessário alterá-los.

3.3. Navegação entre registradores

Como os displays de 7 segmentos disponíveis na placa não são suficientes para exibir os 32 registradores de 32 bits simultaneamente, foi implementado um mecanismo de navegação: um índice de 5 bits (regIndex), que percorre os 32 registradores (0 a 31), é incrementado a cada acionamento do BOTAO-2 e pode ser decrementado quando a chave SW1 está ativada.


```

//controle do índice
reg [4:0] regIndex;
reg btn_reg_prev;
//reg sw_retro_prev;

always @(posedge btn_reg_int or posedge reset_int) begin
    if (reset_int) begin
        regIndex <= 5'd0;
        btn_reg_prev <= 1'b0;
    end else begin
        //BOTA0-2: avança índice
        if (regIndex == 5'd31)
            regIndex <= 5'd0;
        else
            regIndex <= regIndex + 5'd1;
    end
end
end

```

Figure 18. Lógica de avanço e retrocesso do índice de registrador.

O valor do registrador correspondente ao índice atual é lido diretamente do banco de registradores através de uma porta de depuração (debugIndex / debugValor) adicionada ao módulo bancoRegs, permitindo a leitura externa sem interferir na lógica de leitura e escrita já validada na Parte 1.

3.4. Exibição nos displays de 7 segmentos

A placa possui oito displays de 7 segmentos (HEX0 a HEX7), divididos em três grupos de informação:

- **HEX0 a HEX3:** exibem, em hexadecimal, os 16 bits menos significativos do valor armazenado no registrador selecionado;
- **HEX4 e HEX5:** exibem o índice do registrador atualmente selecionado (00 a 1F em hexadecimal);
- **HEX6 e HEX7:** exibem o byte menos significativo do Program Counter (PC), permitindo acompanhar o avanço da execução.

```

seg7 d0(.numero(regValor[3:0]), .display(HEX0));
seg7 d1(.numero(regValor[7:4]), .display(HEX1));
seg7 d2(.numero(regValor[11:8]), .display(HEX2));
seg7 d3(.numero(regValor[15:12]), .display(HEX3));

//HEX4-5: índice do registrador (00 a 1F)
seg7 d4(.numero({3'b0, regIndexFinal[3:0]}), .display(HEX4));
seg7 d5(.numero({3'b0, regIndexFinal[4]}), .display(HEX5));

//HEX6-7: PC em hex
seg7 d6(.numero(pcInsMem[3:0]), .display(HEX6));
seg7 d7(.numero(pcInsMem[7:4]), .display(HEX7));

//LED6: índice do registrador em binário
assign LED6 = {3'b0, regIndexFinal};

```

Figure 19. Instanciação dos decodificadores de 7 segmentos.

Cada display é controlado por uma instância do módulo seg7, responsável por converter um valor de 4 bits no padrão de segmentos correspondente (catodo comum), reaproveitando a mesma lógica para todos os grupos de informação.

3.5. Indicação do índice em LEDs

Além da exibição em hexadecimal nos displays HEX4 e HEX5, o índice do registrador selecionado também é mostrado de forma binária nos LEDs verdes (LEDG) da placa, oferecendo uma segunda forma de acompanhamento visual durante a navegação entre os 32 registradores.

3.6. Pinagem e restrições de projeto

A associação entre os sinais do módulo `cd` e os pinos físicos da FPGA foi definida no arquivo de configurações do Quartus (`fpgat.qsf`), utilizando o dispositivo EP4CE115F29C7 da família Cyclone IV E. Essa etapa foi necessária para que o clock, o reset, os botões de navegação, os displays de 7 segmentos e os LEDs fossem corretamente associados às entradas e saídas físicas da placa utilizada em laboratório.

3.7. Resultados

Após a síntese e a programação da FPGA, foi possível executar o mesmo conjunto de instruções validado na simulação da Parte 1 diretamente na placa, acionando manualmente o clock a cada ciclo. A navegação entre os registradores através do botão e da chave de retrocesso permitiu confirmar, através dos displays de 7 segmentos, que os valores produzidos pelas instruções `add`, `xor`, `addi`, `sll`, `lw`, `sw` e `bne` coincidiam com os resultados obtidos previamente por simulação, validando a correta portabilidade do caminho de dados para hardware. Executando os seguintes comandos:

```
//instrucoes
memInst[0] = 32'b0000000010100000000000010010011; //addi x1, x0, 10
memInst[1] = 32'b00000000101000000000000100010011; //addi x2, x0, 20
memInst[2] = 32'b000000011001000000000000110010011; //addi x3, x0, 100
memInst[3] = 32'b000000000000100011010000000100011; //sw x1, 0(x3)
memInst[4] = 32'b000000000000000011010000100000011; //lw x2, 0(x3)
memInst[5] = 32'b0000000000010100010000000100010011; //addi x2, x2, -5

/*memInst[0] = 32'b00000000000100000000000100110011; // add x2, x0, x1
```

Figure 20. Lista de comandos

Obtemos o resultado abaixo, com o valor do `x2 = F` (15 em decimal), pois adicionamos 20 e tiramos 5:

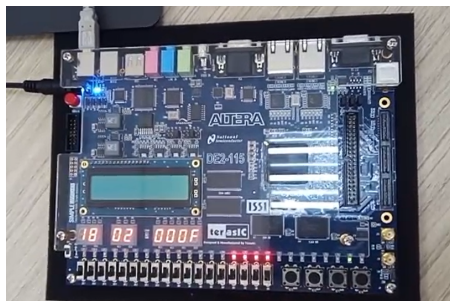


Figure 21. Placa em execução, exibindo o valor do `x2`.

A execução das outras instruções pode ser vista no vídeo do link <https://drive.google.com/file/d/1XawlkULg5YBLTsP4cJ7whK0swWY0e-Vm/view?usp=sharing>.

3.8. Instruções de Execução

Para sintetizar o projeto, o arquivo de projeto `fpgat.qpf` foi aberto no Quartus, e a compilação completa (*Start Compilation*) foi executada a partir do arquivo top-level `cd.v`, gerando o arquivo de programação `fpgat.sof`. Em seguida, utilizando o Programmer do Quartus, o arquivo `.sof` foi carregado na placa através da interface USB-Blaster, permitindo a execução do processador em hardware.

4. Conclusão

A realização deste trabalho permitiu aplicar, na prática, diversos conceitos estudados na disciplina de Organização de Computadores, principalmente relacionados ao funcionamento interno de processadores e à arquitetura RISC-V. Durante o desenvolvimento da Parte 1 foi possível compreender melhor como cada módulo do caminho de dados atua individualmente e como todos trabalham em conjunto para executar as instruções, validando esse comportamento por meio de simulação e da observação dos registradores no terminal.

Na Parte 2, a portabilidade do mesmo caminho de dados para uma placa de FPGA trouxe desafios adicionais, como a adequação dos sinais de clock e reset à lógica dos botões físicos, a criação de um mecanismo de navegação entre os 32 registradores e a exibição de informações em displays de 7 segmentos e LEDs. Essa etapa foi fundamental para compreender, na prática, o processo de síntese de um projeto descrito em Verilog, o planejamento de pinos em uma ferramenta como o Quartus e as diferenças entre validar um circuito por simulação e validá-lo diretamente em hardware.

A implementação dos módulos, do testbench e, posteriormente, da interface física na FPGA também proporcionou uma experiência importante com descrição de hardware, simulação e síntese, exigindo atenção aos detalhes da arquitetura, aos sinais de controle e à comunicação entre os componentes do processador. Além disso, os testes realizados em ambas as etapas ajudaram a entender de forma mais clara o comportamento das instruções e a importância da validação durante todo o desenvolvimento.

Ao longo do trabalho foram encontrados alguns desafios, principalmente relacionados à interpretação das instruções, organização dos sinais de controle, depuração dos erros durante as simulações e, na segunda etapa, ao ajuste dos sinais físicos da placa e à pinagem no Quartus. Entretanto, essas dificuldades contribuíram para aprofundar o aprendizado e desenvolver uma visão mais prática sobre o funcionamento de um processador, desde sua descrição em Verilog até sua execução real em hardware.

Por fim, o trabalho foi fundamental para consolidar os conhecimentos adquiridos em sala de aula, proporcionando uma experiência prática bastante enriquecedora sobre arquitetura de computadores e implementação de sistemas digitais utilizando RISC-V, tanto em simulação quanto em hardware reconfigurável.

5. Referências

RISC-V Foundation. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Disponível em: <https://riscv.org>

Patterson, David A.; Hennessy, John L. *Computer Organization and Design RISC-V Edition*. Morgan Kaufmann, 2017.